IES 9-018 "Gobernador Celso Jaque"

Tecnicatura Superior en Desarrollo de Software

Arquitectura y Diseño de Interfaces — Compensatorio ADI

Diseñar software útil y sostenible en la era de la IA

Criterio humano, evidencia y decisiones de arquitectura, diseño e interfaz

Alumno: Molina Ezequiel

Docente: Paulo Alvarez

Año: 2026

Fecha de entrega: 12 de noviembre de 2026

Introducción

Este informe investiga qué decisiones de arquitectura, diseño e interfaz siguen siendo necesarias cuando la IA acelera la producción de software. El objetivo no es defender una herramienta ni una moda, sino distinguir qué aporta la IA, qué sigue exigiendo criterio humano y cómo aplicar esas conclusiones a un proyecto concreto. Para eso comparé materiales del curso, tres fuentes externas actuales y el caso publicado de Prime Video; después revisé mi propio repositorio y propuse una solución para el sector educativo. Mi postura es que la IA reduce el costo de producir artefactos, pero no elimina la necesidad de límites, contratos, evidencia y responsabilidad.

1. Diseñar hoy

Los equipos organizan una aplicación según cómo esperan cambiarla, quién la mantiene y qué costo operativo toleran. Las alternativas habituales son el **monolito modular**, los **microservicios**, el **serverless** y las arquitecturas en capas o hexagonales. No se eligen por prestigio: el monolito modular suele preferirse cuando el equipo es pequeño y el dominio todavía se está descubriendo; los microservicios, cuando hay equipos autónomos y límites de negocio estables; el serverless, cuando la carga es intermitente y se prioriza no administrar servidores. La decisión depende de acoplamiento, frecuencia de cambio, consistencia de datos, costo de operación y capacidad real del equipo.

El caso de Prime Video es ilustrativo. Su equipo de análisis de calidad de video tenía un servicio distribuido con Step Functions, Lambda y S3. La arquitectura era rápida de construir, pero solo soportaba cerca del 5 % de la carga esperada y el costo de orquestación y de transferencia de datos era alto. Decidieron reunir los componentes en un solo proceso, conservando las mismas responsabilidades, y reportaron una reducción de costos superior al 90 % y mayor capacidad de escala (Kolny, 2023). No demostraron que "el monolito sea mejor": mostraron que la arquitectura debe responder a un caso concreto. La modularidad sigue existiendo aunque el despliegue sea único; lo que cambió fue la unidad de despliegue, no el análisis del dominio.

La IA modifica el costo de explorar alternativas: puede generar prototipos, pruebas, migraciones y documentación en minutos, y eso permite comparar diseños antes. Pero las decisiones que requieren criterio humano son las que comprometen el negocio: dónde poner los límites del dominio, qué contrato exponer, cómo modelar los datos, qué datos personales se guardan, cómo se autoriza una acción, qué se mide y quién responde cuando algo falla. La IA puede proponer una respuesta plausible y equivocada; por eso necesita especificaciones, pruebas, revisión y responsabilidad humana. Thoughtworks (2025, pp. 7-8) observa el avance de agentes y MCP, pero también advierte sobre la "complacencia con el código generado por IA": acelerar no es lo mismo que comprender.

2. Tendencias y debate

La afirmación que elegí es la de Andrej Karpathy (2025): con el "vibe coding" uno "se entrega a las vibraciones, abraza las exponenciales y olvida que el código existe" (traducción propia). Él mismo cuenta que acepta los cambios sin leerlos y que eso funciona para proyectos descartables de fin de semana. La afirmación es una **opinión provocadora** con una predicción implícita: si los modelos mejoran, describir intención podría bastar para construir y modificar productos.

Simon Willison (2025) contrasta esa idea. Sostiene que el vibe coding es valioso para prototipos y experimentos, pero que no todo el desarrollo asistido por IA es vibe coding. Para software que otras personas van a usar, su regla es no incorporar código que no se pueda explicar; si una persona revisó, probó y comprendió lo que generó el modelo, eso es desarrollo de software, no vibe coding. Thoughtworks (2025) aporta una observación de industria: las herramientas y los agentes avanzan rápido, pero la calidad, la seguridad y el mantenimiento siguen dependiendo de juicio humano. El caso Prime Video es **evidencia** de que las decisiones de arquitectura importan; las afirmaciones sobre lo que ocurrirá en los próximos años son **predicciones**, todavía no verificadas.

Las consecuencias para el diseño son concretas. Si la interfaz se vuelve más conversacional o basada en intenciones, el sistema debe mostrar qué entendió, permitir previsualizar y deshacer, pedir confirmación antes de acciones destructivas y registrar quién hizo qué. La arquitectura necesita contratos claros, límites de permisos, observabilidad y presupuestos de costo. La experiencia de usuario no puede depender de que el

modelo "adivine" bien: debe hacer visible la incertidumbre y ofrecer una salida manual. En resumen, la IA puede mediar más tareas, pero eso aumenta la necesidad de interfaces predecibles y de sistemas auditables.

3. Revisar lo aprendido

Del repositorio del curso tomé `01-teoria/06-ia-como-aliado.md`. La idea central es: "La IA no te reemplaza. Te potencia. Pero solo si sabés lo suficiente para revisar lo que produce" (IES 9-018, 2026). Sigue siendo válida y coincide con Willison: la revisión y la comprensión son la frontera entre asistencia responsable y delegación ciega. Requiere una actualización: el archivo habla de un equipo de agentes y de revisar sus salidas, pero convendría distinguir explícitamente prototipo, producción y sistemas regulados, y agregar la verificación de fuentes y de resultados, no solo del código. También debería decir que la responsabilidad final es humana aunque el artefacto lo haya generado un agente.

Del cuadernillo tomé el TP1, "De Vibe Coding a SDD" (IES 9-018, 2026, pp. 7-8). Allí la SPEC se define como contrato del sistema, con requisitos numerados y Non-Goals, y el arnés fija reglas para el agente. La idea sigue siendo útil y se refuerza con el debate actual: Thoughtworks (2025, p. 8) discute el desarrollo guiado por especificaciones como práctica emergente, pero advierte el riesgo de una especificación inicial excesiva. Por eso el TP1 no queda obsoleto: se aplica mejor si la SPEC es un artefacto vivo, con criterios de aceptación y trazabilidad, y si el arnés se revisa junto con el código. En proyectos pequeños o descartables puede alcanzar una versión más liviana; en producción, la especificación y las pruebas son barreras necesarias. Una novedad no invalida todo lo anterior: cambia el énfasis.

4. Mirar tu propio proyecto

Elegí el flujo de reserva y la agenda administrativa de mi Sistema de Turnos. La organización se entiende: el README describe el stack y la estructura; los ADR registran las decisiones de arquitectura; `docs/arquitectura/api-contracts.yaml` define el contrato antes de la implementación; el servidor separa rutas, middleware y reglas; el cliente separa componentes y servicios de API. Es un monolito modular, no un conjunto de piezas acopladas por accidente. Esto permite modificar una parte sin rehacer todo: por ejemplo, `server/middlewares/auth.ts` centraliza la autenticación y `requireRole` protege las rutas administrativas, mientras las reglas de negocio y los tests permiten cambiar una validación sin tocar la interfaz.

La interfaz ayuda a resolver la tarea. `client/src/components/AdminAgenda.tsx` permite elegir una fecha, ordena los turnos por hora y muestra el estado con ícono, texto y color, además de estados de carga, error y lista vacía; también usa `aria-busy` y etiquetas asociadas. No obliga a memorizar códigos ni a interpretar solo un color. En el flujo de reserva, la selección de servicio, profesional y horario guía al usuario, y los mensajes de error intentan explicar qué corregir.

Conservaría el monolito modular, el contrato primero, los ADR, la separación de responsabilidades y el pipeline de CI. La mejora que elegiría es completar el contrato OpenAPI para cubrir el 100 % de las rutas, incluidos el CRUD administrativo de servicios y profesionales y `POST /auth/register`, y generar tipos y pruebas de contrato desde ese archivo. Hoy el contrato v1.1.0 documenta seis operaciones críticas, pero el resto de la API quedó fuera; es una deuda de documentación reconocida en `docs/informe-final.md` (sección 10). El esfuerzo es medio: hay que escribir esquemas, validar con lint y reemplazar tipos manuales. La necesidad es real: evita que la implementación y la documentación se separen, facilita el onboarding y da a los agentes un límite verificable. No es una moda: es una decisión que reduce errores.

5. Adaptarse a un rubro

Elegí el sector educativo. La necesidad es la retroalimentación formativa: en escuelas técnicas, un docente con muchos estudiantes tarda en devolver correcciones detalladas, y el estudiante recibe la devolución cuando ya avanzó. Una solución posible es un **asistente de retroalimentación formativa** para docentes: el profesor carga la consigna y la rúbrica; la IA propone comentarios por estudiante citando criterios de la rúbrica; el docente revisa, edita o descarta; recién entonces el estudiante ve la devolución. No es un chatbot libre para menores ni un reemplazo del docente: es una herramienta con aprobación humana.

La usarían docentes como usuarios principales, estudiantes como destinatarios y coordinadores para supervisar. La interacción sería estructurada: formulario de rúbrica, vista lado a lado de la propuesta y el trabajo, botones de aprobar/editar/rechazar, y una vista del estudiante con el criterio, la observación y un próximo paso. Para mantenerla habría que versionar rúbricas y prompts, conservar un conjunto de

evaluaciones corregidas, medir la calidad de las sugerencias, abstraer el proveedor de modelo y limitar el costo por curso. En datos: minimización, consentimiento informado, cifrado, control de acceso por rol, registro de auditoría, política de retención y prohibición de entrenar con trabajos de estudiantes. Para adaptarla, plantillas por materia, exportación a CSV/LTI y módulos reemplazables.

La propuesta se diferencia de mi proyecto implementado: el Sistema de Turnos agenda citas y no procesa producciones de estudiantes; comparte patrones (monolito modular, contrato, roles, tests), pero el dominio, los datos y el riesgo son distintos. También se diferencia de un "chatbot educativo" genérico porque el valor está en el flujo de revisión docente y en la trazabilidad de la rúbrica, no en la conversación. La tendencia de agentes e interfaces por intención es útil aquí solo con guardrails: el docente decide y el sistema deja evidencia.

Conclusiones

Aprendí que la IA cambia la economía de producir software, pero no reemplaza las decisiones que sostienen un producto: límites, contratos, seguridad, datos, operación y responsabilidad. La decisión que revisaría primero en mi proyecto es ampliar el contrato al 100 % de las rutas y generar tipos y pruebas desde él, porque responde a una necesidad concreta y reduce el riesgo de que la documentación quede atrás. Lo que sigue incierto es la confiabilidad de los agentes, el costo real a escala, la regulación sobre datos de menores y la forma de evaluar la calidad de las sugerencias. Por eso prefiero medir antes de generalizar: la IA como acelerador, no como excusa para dejar de entender lo que construimos.

Referencias

1. IES 9-018. (2026). *La IA como aliado de aprendizaje* [archivo `01-teoria/06-ia-como-aliado.md`]. Repositorio de Arquitectura y Diseño de Interfaces. https://github.com/IES9018/proyecto-adi-2026
2. IES 9-018. (2026). *Cuadernillo del proyecto ADI*, TP1 "De Vibe Coding a SDD", pp. 7-8. Repositorio de Arquitectura y Diseño de Interfaces.
3. Karpathy, A. (2025, febrero). *Vibe coding* [publicación en X]. Reproducida en Willison, S. (2025, 6 de febrero), *A quote from Andrej Karpathy*. https://simonwillison.net/2025/Feb/6/andrej-karpathy/
4. Kolny, M. (2023, 22 de marzo). *Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%*. Prime Video Tech. https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90 (copia archivada: https://archive.is/h788m)
5. Thoughtworks. (2025, noviembre). *Technology Radar, Vol. 33*, pp. 7-8. https://www.thoughtworks.com/content/dam/thoughtworks/documents/radar/2025/11/tr_technology_radar_vol_33_en.pdf
6. Willison, S. (2025, 19 de marzo). *Not all AI-assisted programming is vibe coding (but vibe coding rocks)*. https://simonwillison.net/2025/Mar/19/vibe-coding/

Declaración de uso de IA

> Herramienta: OpenCode (asistente de IA en terminal). Uso: apoyo para ordenar el borrador, revisar redacción y detectar incoherencias. La verificación de las fuentes, la selección de los ejemplos, la postura final y la responsabilidad sobre el contenido son del autor.